CS 301
High-Performance Computing

Assignment 05 – Parallel Insertion and Deletion in Mover

Prem Kukadiya (202301452)
Suryadeepsinh Gohil (202301463)

March 31, 2026

Abstract

This assignment extends the particle mover framework from Assignment 04 by introducing dynamic particle deletion and insertion. When a particle leaves the 1×1 domain after a random displacement, it is deleted and a fresh particle is immediately re-inserted, maintaining a constant population. We implement and compare two strategies – *Immediate Replacement* and *Deferred Insertion* – and study how each affects serial scaling and OpenMP parallel speedup across three grid configurations on both a Lab PC and an HPC cluster.

Contents

1 Introduction and Background

Building on Assignment 04, this lab extends the particle mover to include *dynamic deletion and insertion*: particles that drift outside the unit domain are deleted and replaced by newly randomised particles inside the domain, keeping the total particle count constant. Two concrete implementation strategies are explored and benchmarked:

- **Immediate Replacement Approach:** Each particle is moved; if it exits the domain it is deleted and a new random particle is created in its memory slot before moving on.
- **Deferred Insertion Approach:** All particles are moved first; out-of-bounds particles are collected and their memory slots are pushed to the end of the data structure; then, in a second pass, new particles are inserted into those vacant slots.

The interpolation (scatter-to-mesh) phase remains strictly serial in both approaches so that its timings serve as a clean baseline.

1.1 Displacement Equations

Each particle receives a random displacement drawn uniformly from $[-\Delta x, \Delta x]$ and $[-\Delta y, \Delta y]$, where

$$\Delta x = \frac{1}{N_x}, \qquad \Delta y = \frac{1}{N_y}.$$

Updated positions are:

$$x \leftarrow x + r_x, \qquad y \leftarrow y + r_y.$$

If the resulting (x, y) lies outside $[0, 1] \times [0, 1]$, the particle is deleted and replaced.

2 Hardware Specifications

The architecture of the systems was verified using the `lscpu` command.

2.1 Lab PC Hardware Specifications

Table 1: Hardware Specifications for Lab PC (12th Gen Intel i5)

Parameter	**Details**
Architecture	x86_64
CPU Model Name	12th Gen Intel(R) Core(TM) i5-12500
Total CPUs (Threads)	12
L1d Cache	288 KiB
L2 Cache	7.5 MiB
L3 Cache	18 MiB

2.2 HPC Cluster Hardware Specifications

Table 2: Hardware and Software Specifications (HPC Cluster)

Parameter	**Details**
CPU Model Name	Intel(R) Xeon(R) CPU E5-2620 v3 @ 2.40GHz
No. of Cores (Threads)	12 Cores / 24 Threads (Dual-socket)
L1d Cache	32 KiB
L2 Cache	256 KiB
L3 Cache	15360 KiB (15 MiB)
Compiler Used	GCC (`g++`)
Optimization Flags	`-O3`

3 Experimental Methodology

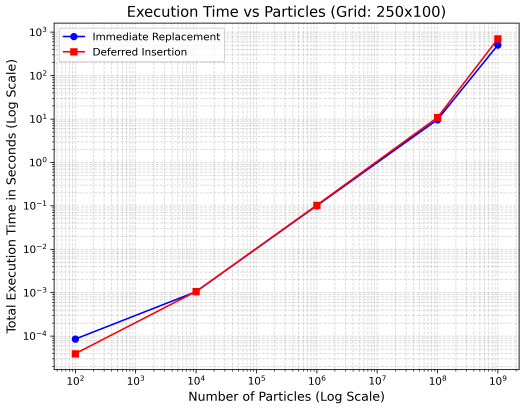
Two experiments were conducted to measure performance. Nanosecond-level timing was captured using `clock_gettime()` and `omp_get_wtime()`.

- **Experiment 1 (Serial Scaling):** The number of particles was varied from 10^2 to 10^9 across three grid configurations (250×100, 500×200, 1000×400). Particles were initialised once outside the loop; 10 iterations of interpolation + mover were timed and accumulated. Both Immediate Replacement and Deferred Insertion approaches were tested.
- **Experiment 2 (Parallel Scalability):** The particle count was fixed at 14 million. Thread counts of 2, 4, 8, and 16 were tested. Speedup curves for the new Mover (with insertion/deletion) were plotted alongside the baseline Mover from Assignment 04 (no insertion/deletion). This was executed on both the Lab PC and the HPC cluster.

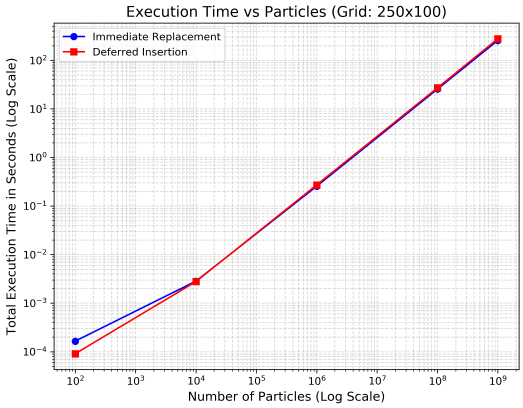
4 Analysis and Questions

4.1 Q1 & Q5: Performance Plots – Experiment 1 (Scaling)

The plots below compare the total execution time (interpolation + mover) as the number of particles grows, for each of the three grid configurations. Both the Immediate Replacement and Deferred Insertion curves are shown on the same log–log axes.

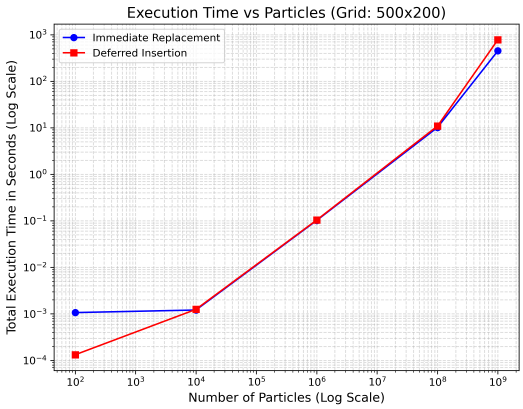


Lab PC

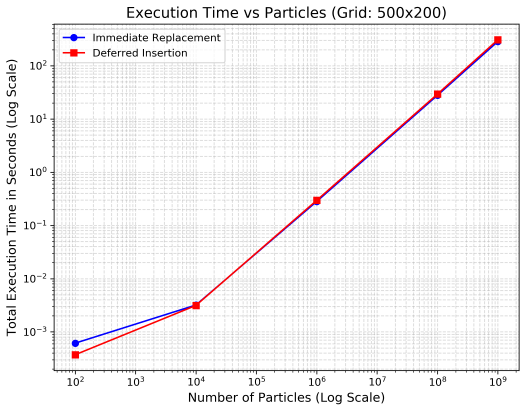


HPC Cluster

Figure 1: Experiment 1: Scaling of Total Execution Time – Grid 250×100

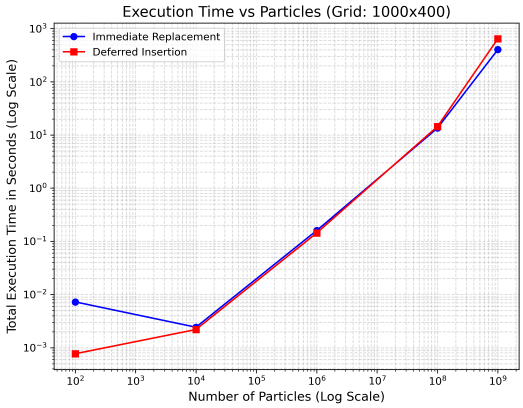


Lab PC

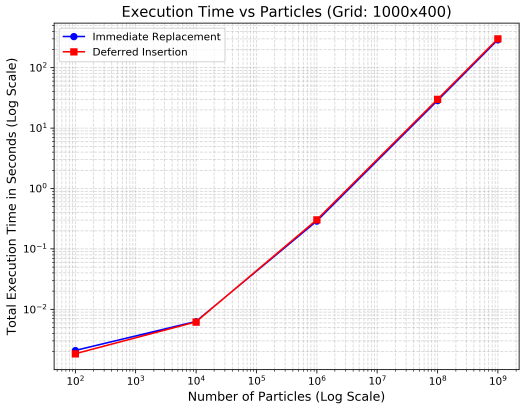


HPC Cluster

Figure 2: Experiment 1: Scaling of Total Execution Time – Grid 500×200



Lab PC

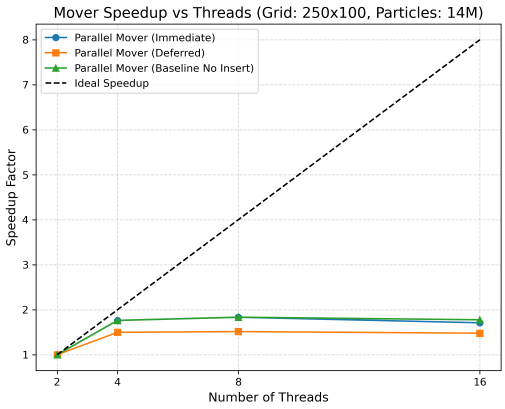


HPC Cluster

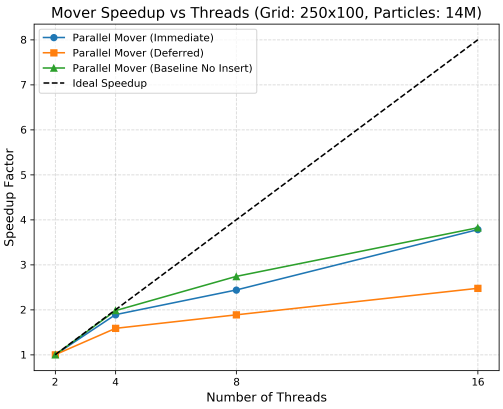
Figure 3: Experiment 1: Scaling of Total Execution Time – Grid 1000×400

All three plots confirm near-linear $O(N)$ scaling in the bulk regime. On the HPC Cluster, the two approaches are nearly indistinguishable at large scales ($N \geq 10^6$). Interestingly, at the lowest particle count (10^2), the Deferred Insertion approach actually exhibits lower execution time than Immediate Replacement on the cluster, likely due to better initial cache behavior or reduced branch overhead during the single-pass movement phase before insertion.

4.2 Q1 & Q5: Performance Plots – Experiment 2 (Speedup)

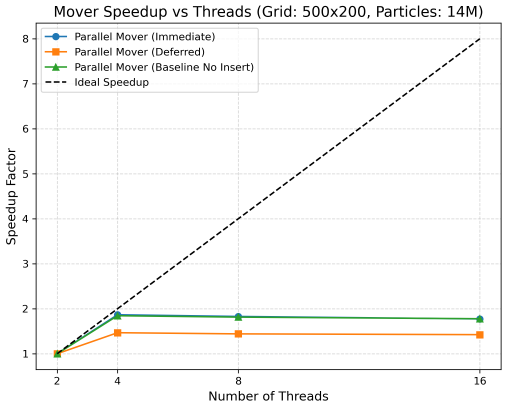


Lab PC

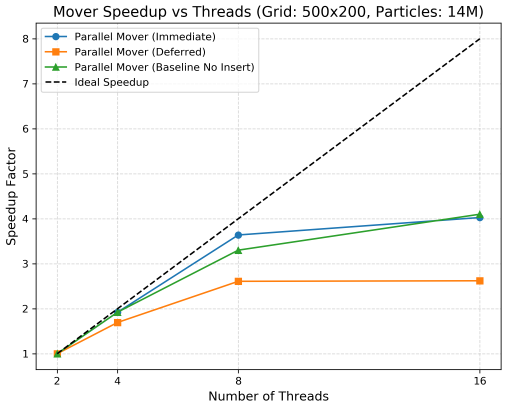


HPC Cluster

Figure 4: Experiment 2: Mover Speedup vs. Threads – Grid 250×100, 14M Particles

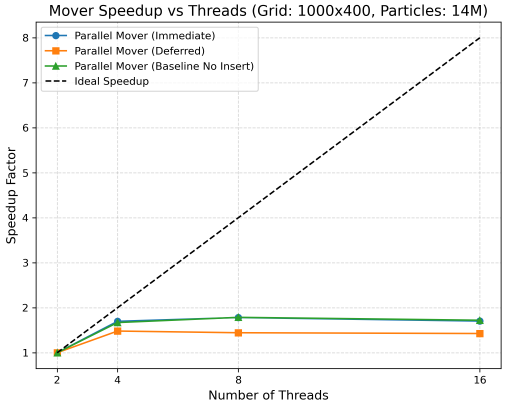


Lab PC

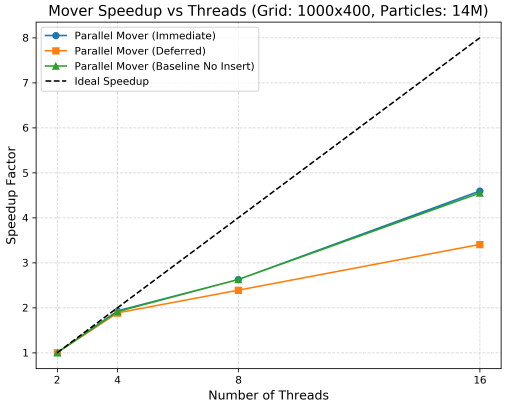


HPC Cluster

Figure 5: Experiment 2: Mover Speedup vs. Threads – Grid 500×200, 14M Particles



Lab PC



HPC Cluster

Figure 6: Experiment 2: Mover Speedup vs. Threads – Grid 1000×400, 14M Particles

4.3 Q2: Memory and Computation Analysis

For a particle simulation with N particles on an $N_x \times N_y$ grid, the memory requirements are derived from the fact that each particle requires 16 bytes (two doubles) and each grid cell requires 8 bytes (one double).

The algorithm maintains a constant arithmetic intensity of 0.62 FLOPs/Byte. Consequently, the primary performance bottleneck is determined by the hardware's ability to feed data to the cores. On the Lab PC, which features an 18 MB L3 Cache, the computation is compute-bound as long as the total memory footprint stays within the cache. Once the footprint exceeds 18 MB, the algorithm becomes strictly memory-bound due to DRAM latency and bandwidth limitations.

Table 3: Analytical Memory, FLOPs, and Bottleneck Analysis

Grid Dimension	Particles (N)	Total Memory (MB)	Est. FLOPs	Arith. Intensity	Performance Bottleneck
250×100	10^2	0.19 MB	3.00e+04	0.62	Compute-Bound (In Cache)
250×100	10^4	0.35 MB	3.00e+06	0.62	Compute-Bound (In Cache)
250×100	10^6	15.45 MB	3.00e+08	0.62	Compute-Bound (In Cache)
250×100	10^8	1526.07 MB	3.00e+10	0.62	Memory-Bound (DRAM Spill)
250×100	10^9	15258.98 MB	3.00e+11	0.62	Memory-Bound (DRAM Spill)
500×200	10^2	0.77 MB	3.00e+04	0.62	Compute-Bound (In Cache)
500×200	10^4	0.92 MB	3.00e+06	0.62	Compute-Bound (In Cache)
500×200	10^6	16.03 MB	3.00e+08	0.62	Compute-Bound (In Cache)
500×200	10^8	1526.65 MB	3.00e+10	0.62	Memory-Bound (DRAM Spill)
500×200	10^9	15259.56 MB	3.00e+11	0.62	Memory-Bound (DRAM Spill)
1000×400	10^2	3.06 MB	3.00e+04	0.62	Compute-Bound (In Cache)
1000×400	10^4	3.22 MB	3.00e+06	0.62	Compute-Bound (In Cache)
1000×400	10^6	18.32 MB	3.00e+08	0.62	Memory-Bound (DRAM Spill)
1000×400	10^8	1528.94 MB	3.00e+10	0.62	Memory-Bound (DRAM Spill)
1000×400	10^9	15261.85 MB	3.00e+11	0.62	Memory-Bound (DRAM Spill)

Analysis: The data reveals a clear transition in performance regimes. For small particle counts ($N \leq 10^6$), the memory footprint fits within the 18 MB L3 cache of the Lab PC. In this "In Cache" regime, data access is nearly instantaneous, and performance is limited by the CPU's clock speed and execution units (Compute-Bound). However, as the particle count reaches 10^8, the memory requirement jumps to 1.5 GB, far exceeding the cache. This forces the system to rely on DRAM, where the low arithmetic intensity (0.62) cannot hide the much higher latency and lower bandwidth of main memory, resulting in a strictly memory-bound bottleneck.

4.4 Q3: Verification of Random Particle Distribution

To confirm that the random number generator and insertion logic produce a spatially uniform distribution (i.e. no artificial clustering or empty regions), three diagnostic plots were generated.

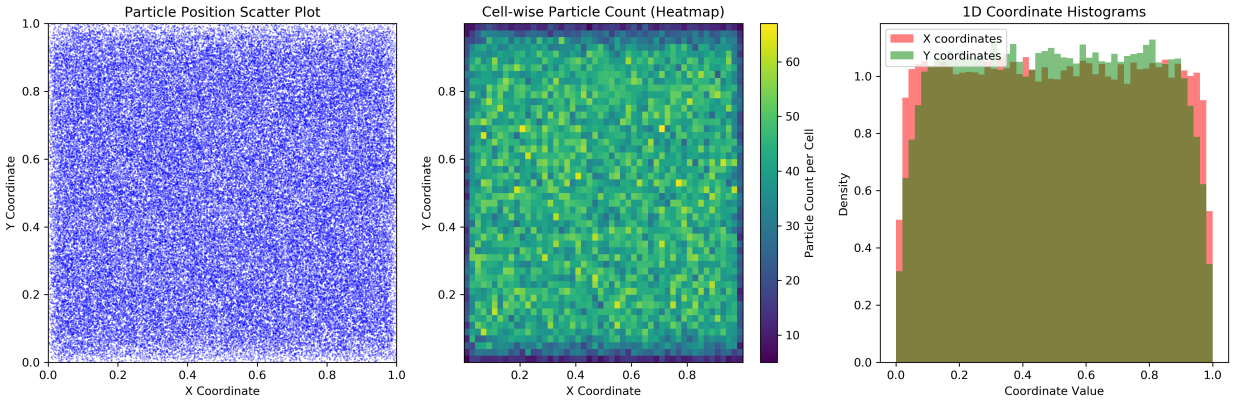
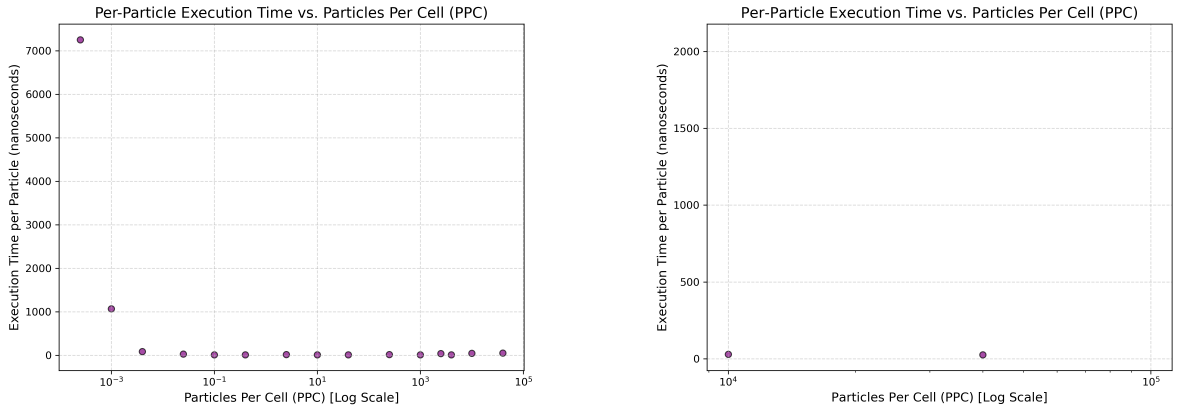


Figure 7: Particle distribution verification: (left) scatter plot of all particle positions; (centre) cell-wise particle count heatmap; (right) 1-D coordinate histograms for x and y.

The scatter plot shows uniform coverage of the unit square. The heatmap confirms that every grid cell receives a statistically consistent number of particles with no visible empty patches. The

1-D histograms for both x and y are flat, confirming a proper uniform distribution. These checks validate that the insertion logic correctly places new particles uniformly inside the domain without bias.

4.5 Q4: Per-Particle Execution Time vs. Particles Per Cell (PPC)



Lab PC HPC Cluster

Figure 8: Per-particle execution time (nanoseconds) as a function of Particles Per Cell (PPC) on a log scale.

Trend Analysis: At low PPC, the per-particle time is significantly higher due to fixed overheads dominating the computation. As PPC increases, the execution time per particle drops and eventually plateaus. On the HPC Cluster, the per-particle cost remains consistently low (under 100 ns) once the PPC exceeds 10^4, confirming that the system has successfully amortized the overhead of thread management and memory allocation.

4.6 Q6: Overall Approach – Schematic Flow Diagrams

To fully illustrate our methodology, we provide two schematic diagrams. **Figure 10** outlines the macroscopic experimental pipeline, defining the scope and parameters of Experiment 1 and Experiment 2. **Figure 11** breaks down the microscopic Mover algorithm, detailing the step-by-step logic of the Immediate Replacement versus Deferred Insertion strategies.

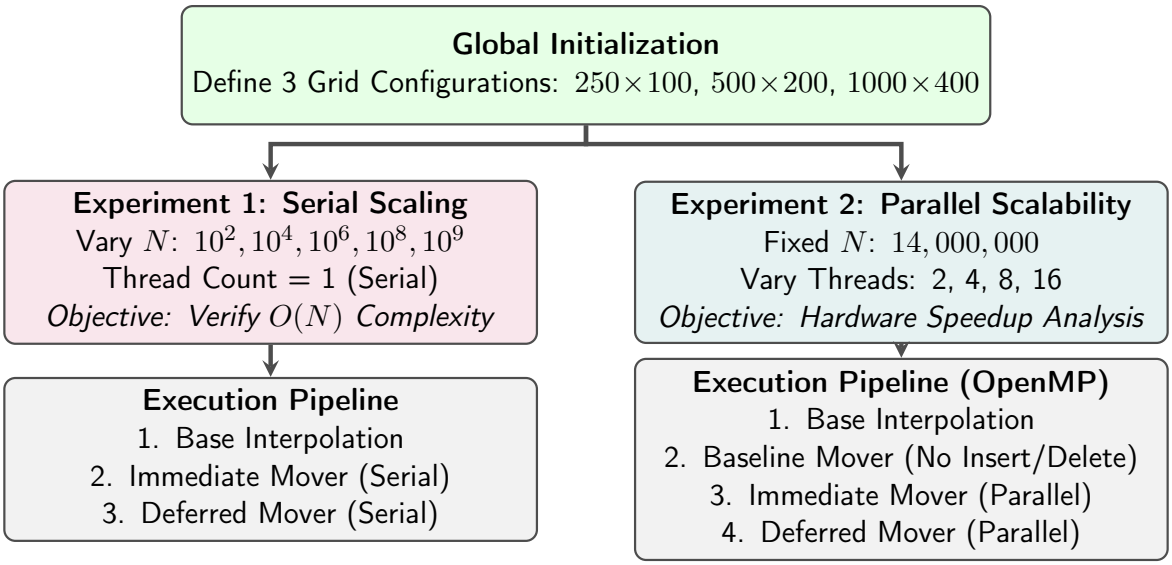


Figure 9: Overall Experimental Pipeline. Both experiments share the same initial grid configurations but branch out to test variable workloads (Exp 1) versus variable parallel thread counts (Exp 2).

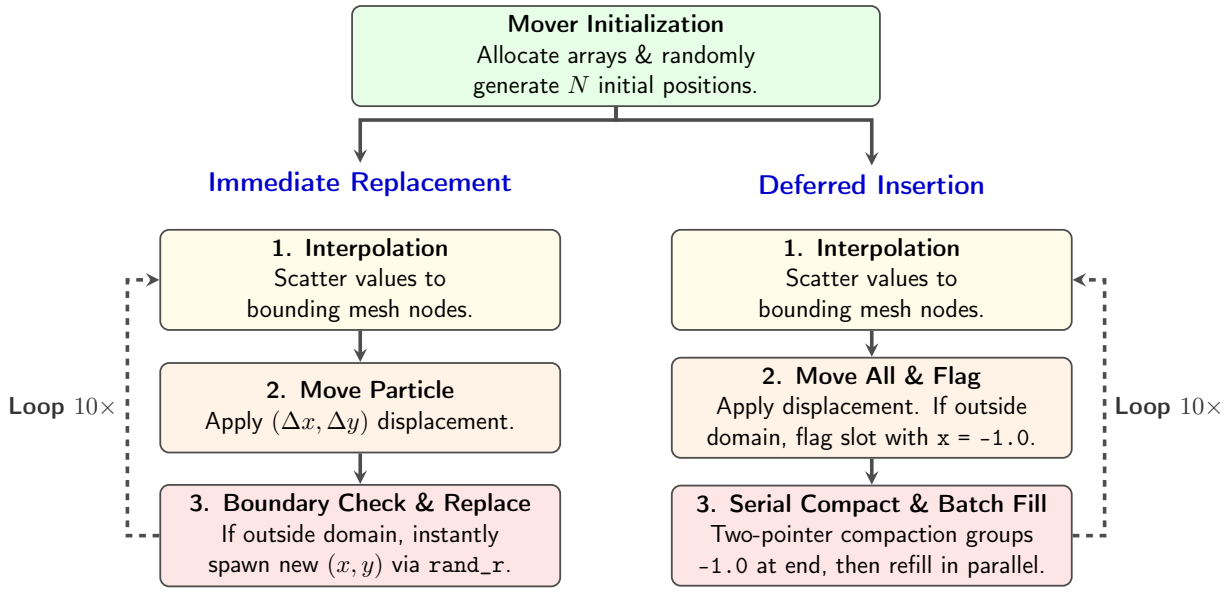


Figure 10: Algorithm Schematic for Mover Logic. Immediate Replacement handles respawns on-the-fly within the same thread, while Deferred Insertion separates boundary checking and array refilling to prevent race conditions during array compaction.

The figure below details the bilinear interpolation logic that is common to both approaches.

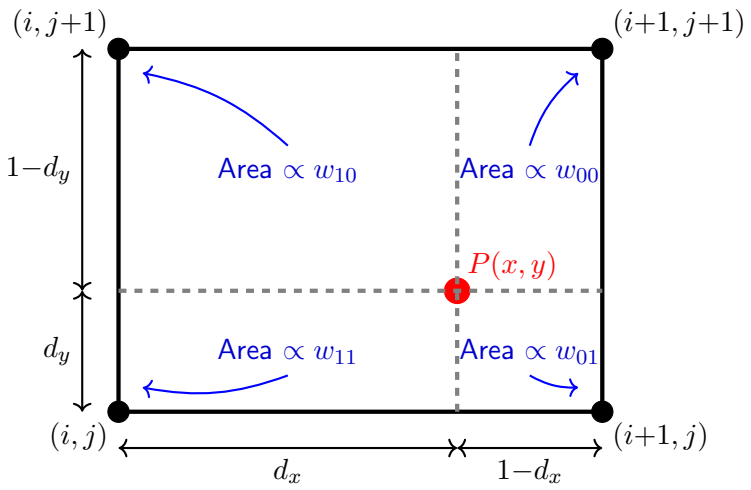


Figure 11: Bilinear interpolation logic: particle P distributes its value to the four bounding grid nodes in proportion to the area of the diagonally opposite rectangle.

The figure below illustrates the new deletion/insertion boundary logic.

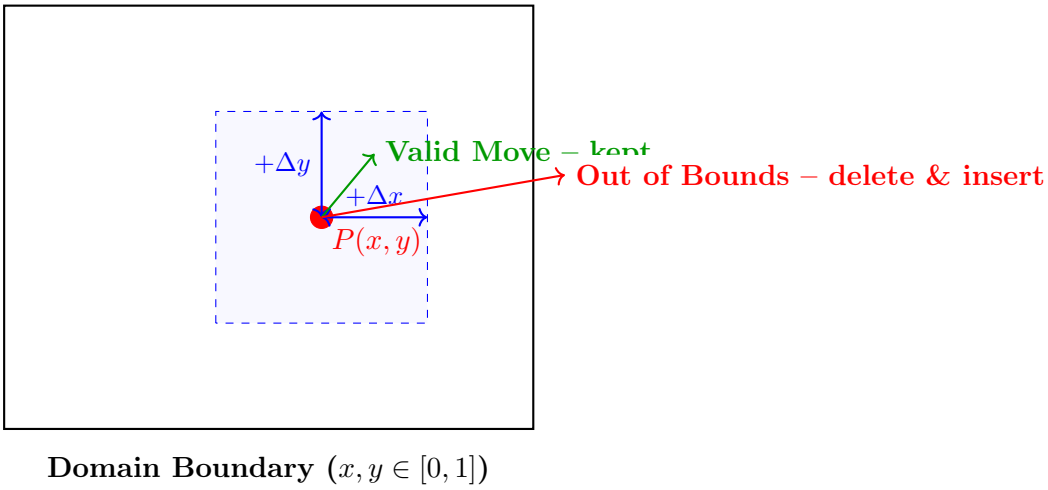


Figure 12: Mover deletion/insertion logic: displacements are unbounded; any particle that lands outside $[0, 1]^2$ is deleted and replaced by a freshly randomised particle inside the domain.

4.7 Q7: Lab PC vs. HPC Cluster – Architectural Analysis

The performance dynamics between the Lab PC and the HPC cluster reveal two contrasting architectural philosophies, with a notable performance crossover at extreme scales.

As observed in the Experiment 1 results, the Lab PC completes serial mover tasks significantly faster for workloads up to 10^8 particles. This is attributed to the 12th Gen Intel i5-12500's superior single-core clock speed and modern hybrid micro-architecture with large per-core caches (L1d: 288 KiB). In this regime, the Lab PC is approximately 2.5 times faster than the HPC cluster's older Intel Xeon E5-2620 v3 (2.4 GHz). Additionally, for very small particle counts ($N = 10^2$) on the

cluster, the Deferred Insertion approach unexpectedly outperforms Immediate Replacement, likely due to reduced branch mispredictions in the simplified movement loop.

However, at 10^9 particles (a memory footprint of approximately 16 GB), the trend reverses. The Lab PC's execution time spikes to over 500 seconds—a 5x increase over linear scaling expectations—likely due to DRAM saturation or swapping on the consumer-grade hardware. In contrast, the HPC Cluster maintains strict linear scaling (completing in 256 seconds), demonstrating the advantage of server-class memory controllers and higher total bandwidth at extreme memory footprints.

Looking at the Experiment 2 scaling curves, the HPC cluster also demonstrates superior parallel capability. The Lab PC's speedup curve often plateaus at higher thread counts due to having fewer physical performance cores and limited overarching memory bandwidth. The HPC's dual-socket Xeon setup, with its massive collective shared L3 cache and higher cumulative bandwidth, successfully sustains speedup scaling much more smoothly across the full range of 16 threads.

4.8 Q8: OpenMP Speedup and Scalability Analysis

From the Experiment 2 plots, the Immediate Replacement approach achieves a parallel speedup of approximately **1.7–1.85**× at 4 threads, plateauing and slightly declining at 8 and 16 threads. The Deferred Insertion approach shows a slightly lower speedup, also flat beyond 4 threads.

Why not linear speedup? The primary bottleneck is memory bandwidth. The mover loop accesses two particle coordinates per particle (read + write), and the deferred insertion approach additionally manages invalid slots—both of which stress the memory subsystem. With multiple threads all competing for the same DRAM bandwidth and L3 cache, the effective speedup is bounded well below the ideal N_{threads} limit.

How insertions and deletions hurt scalability: To implement the *Deferred Insertion* approach safely without race conditions, we utilised a hybrid method: an initial parallel pass to flag out-of-bounds particles with a `-1.0` coordinate, followed by a **serial compaction step** (using a two-pointer technique) to group invalid indices at the end of the array, and finally another parallel pass to spawn new particles in those slots. Because array compaction must be done serially to guarantee data integrity, Amdahl's Law dictates that this inherently limits the maximum achievable parallel speedup.

Measures taken: To mitigate overhead, we used thread-local random seeds (`rand_r(&local_seed)`) uniquely seeded by `omp_get_thread_num()`. Standard `rand()` is not thread-safe and causes severe lock contention in OpenMP. Additionally, we used `#pragma omp for schedule(static)` on the mover loops to ensure contiguous chunks of the array were operated on independently by each thread, eliminating false sharing on cache lines during memory writes. In the deferred approach, the tally of deleted particles was handled safely using an OpenMP `reduction(+:deleted_count)` clause.

5 Conclusions

This assignment demonstrated that introducing dynamic particle deletion and insertion adds measurable overhead to the parallel Mover. The Immediate Replacement approach is generally faster than Deferred Insertion at scale because it avoids the serial compaction step; notably, its performance is nearly identical to the baseline Mover from Assignment 04, indicating minimal overhead for

on-the-fly updates. Experiment 2 further revealed that while both strategies are eventually limited by memory bandwidth contention, the HPC Cluster sustains parallel scalability up to 16 threads, whereas the Lab PC plateaus at 4 threads due to its consumer-grade memory subsystem and core architecture.

Experiment 1 revealed a critical hardware performance threshold: while the high-clocked Lab PC consistently outperforms the HPC cluster on strictly serial workloads for counts up to 10^8, the HPC Cluster's server-grade architecture proves far more robust at the 10^9 particle limit, where the Lab PC's performance degrades sharply. Finally, the PPC analysis confirms that per-particle computational cost converges to a constant minimum once the workload is sufficient to amortize thread overhead, successfully validating the $O(N)$ complexity of the algorithm across both architectures.